Terminal ja käsurida

11IT · Veebirakendused · Tööriistad · konspekt

Miks kirjutada, kui saab klõpsata?

Viimases kahes tunnis kirjutasid terminali asju nagu `npm create vite@latest` ja `git commit -m`. Need töötasid, aga ausalt: kas oskaksid öelda, mida iga sõna tähendab? Enamik kirjutas need lihtsalt juhendist maha nagu loitsu. Täna võtame loitsud lahti.

Terminal on vestlus arvutiga. Sina kirjutad käsu, vajutad Enterit ja arvuti vastab tekstiga. Pole nuppe ega menüüsid. Esimese hooga tundub see vanamoodne. Miks ei võiks kõike hiirega teha?

Põhjuseid on mitu. Programmeerija tööriistadel (Node, npm, Git) sageli polegi nuppe, sest nende tegijatel on lihtsam teha üks käsk kui terve aken. Käsu saab kirja panna ja teisele saata: juhendis olev rida töötab igaühe arvutis ühtemoodi, ekraanipilt „vajuta siia“ aga ei pruugi. Ja kõige tähtsam: server andmekeskuses ei ole ekraani ega hiirt. Kui sinu backend kunagi päriselt internetti läheb, räägid sa temaga just nii.

Kus ma olen?

Ava VS Code'is terminal (**Terminal** → **New Terminal**). Kõige alumisel real on midagi sellist:

```
PS C:\Users\opilane\projektid>
```

`PS` tähendab, et see on **PowerShell**, Windowsi terminal. Pärast seda tuleb kaust, kus sa parasjagu oled, ja märk `>` ütleb, et terminal ootab käsku. Kõik, mida kirjutad, tehakse selles kaustas. Mõtle sellest kui failihalduri aknast, mis on ühe kausta peal lahti: kui küsid faili, mida seal kaustas pole, siis seda ka ei leita.

> **Kõige sagedasem viga terminalis ei ole vale käsk, vaid vale kaust.** Kui midagi ei tööta, vaata kõigepealt, kus sa oled. Mina teen seda ka.

Käsu ehitus

Iga käsk on üles ehitatud ühtemoodi: kõigepealt programmi nimi, siis see, millega ta midagi teeb, ja vahel veel lipud, mis muudavad ta käitumist. Osad on eraldatud tühikuga.

```
git      commit      -m "Lisasin vormi"
programm alamkäsk    lipp ja selle väärtus
```

Lipud algavad kriipsuga: lühike kuju ühe kriipsuga (`-m`), pikk kahega (`--template`). Kui väärtuses on tühikuid, pane see jutumärkidesse, muidu arvab terminal, et need on eraldi sõnad. Nüüd saad ka esimese tunni pika loitsu lahti võtta:

```
npm create vite@latest minu-projekt -- --template react
```

`npm` on programm, `create` tema alamkäsk, `vite@latest` ütleb, millist malli kasutada (uusim versioon), `minu-projekt` on kausta nimi ja `--template react` on lipp, mis valib Reacti. Kaks kriipsu üksinda keskel tähendab „edasised lipud ei ole npm-i, vaid Vite'i omad“.

Teed: kuidas kaustu kirja panna

Kausta asukohta nimetatakse **teeks** (ingl *path*). Täielik tee algab kettast, näiteks `C:\Users\opilane\projektid\sookla`. Seda on pikk kirjutada, nii et tavaliselt kasutame **suhtelist** teed, mis algab sealt, kus sa praegu oled. Kolm märki tasub pähe õppida:

Märk	Tähendus	Näide
`.`	see kaust, kus olen	`code .` avab praeguse kausta VS Code'is
`..`	üks kaust ülespoole	`cd ..` liigub ülemisse kausta
`~`	sinu kodukaust	`cd ~` viib koju, kus ka ei oleks

Neid saab kombineerida. `cd ../teine-projekt` tähendab: mine üks samm üles ja sealt kausta `teine-projekt`. Windows kasutab kaldkriipsu `\`, Mac ja Linux `/`, aga PowerShell saab mõlemast aru.

> **Ära pane projekti kausta nimesse tühikuid ega täpitähti.** `Minu Projekt` tuleb iga kord jutumärkidesse panna ja mõni tööriist komistab `söökla` peal. Kirjuta `sookla-arvustused`, väikeste tähtedega ja kriipsudega.

Põhikäsud

Need kümme käsku katavad suurema osa sellest, mida veebiarendaja terminalis failidega teeb. Pane tähele, et paljud nimed on samad mis Macis ja Linuxis. See pole juhus: PowerShell tunneb neid nimesid meelega, sest suurem osa internetis olevaid juhendeid on kirjutatud Maci või Linuxi jaoks.

PowerShell	Mida teeb	Mac / Linux
`pwd`	näitab, kus kaustas oled	sama
`ls`	näitab, mis selles kaustas on	sama
`cd kaust`	liigub kausta sisse	sama
`cd ..`	liigub ühe kausta võrra üles	sama
`mkdir server`	teeb uue kausta	sama
`ni server.js`	teeb uue tühja faili (*New-Item*)	`touch server.js`
`cat fail.txt`	näitab faili sisu	sama
`cp a.txt b.txt`	kopeerib faili (kausta puhul `cp -r`)	sama
`mv a.txt b.txt`	nimetab ümber või tõstab teise kausta	sama
`rm fail.txt`	kustutab faili (kausta puhul `rm -r`)	sama
`cls` või `clear`	puhastab ekraani	`clear`
`code .`	avab praeguse kausta VS Code'is	sama

> `rm` **ei pane faile prügikasti.** Need on kohe läinud ja küsimust „kas oled kindel?“ ei tule. Enne `rm -r` vaata alati üle, kus sa oled.

Kui terminali menüüst valisid kunagi **Command Prompti** (cmd), siis seal on teised nimed: `dir` on `ls` asemel, `type` on `cat` asemel ja `del` on `rm` asemel. `cd`, `mkdir` ja kõik `npm`- ja `git`-käsud töötavad ühtemoodi.

Neli klahvi, mis säästavad poole tunnist

Klahv	Mis juhtub
Tab	lõpetab kausta või faili nime. Kirjuta `cd sook` ja vajuta Tab. Kui midagi ei juhtu, siis sellist nime ei olegi, ja see on samuti kasulik teadmine.
↑	toob tagasi eelmise käsu. Mitu korda vajutades jõuad varasemate juurde.
Ctrl+C	peatab programmi, mis parasjagu töötab, näiteks serveri. Kui tekst on hiirega valitud, siis hoopis kopeerib.
Ctrl+V	kleebib. Paremklõps terminalis teeb sama.

Terminaliakna paremas ülanurgas on **+**, mis avab uue terminali, ja prügikast, mis paneb terminali kinni koos kõigega, mis seal töötas. Prügikast on serveri peatamiseks liiga jõuline viis: selleks on Ctrl+C.

Projekti käsud lahti seletatult

Node ja npm

Node.js on programm, mis käivitab JavaScripti väljaspool brauserit. **npm** tuleb Node'iga kaasa ja tõmbab internetist teiste kirjutatud koodi ehk **pakke**. Kas mõlemad on olemas, saab kontrollida nii:

```
node -v
npm -v
```

Projekti südames on fail `package.json`. Seal on kirjas, milliseid pakke projekt vajab ja millised käsud tal on:

```
{
  "name": "sookla-arvustused",
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  },
  "dependencies": {
    "react": "^19.1.0"
  }
}
```

Käsk	Mida teeb
`npm install`	loeb `package.json`-i ja tõmbab kõik pakid kausta `node_modules`. Seda on vaja siis, kui said projekti kelleltki teiselt või Gitist.
`npm install express`	lisab projekti uue paki ja kirjutab selle `package.json`-i.
`npm run dev`	otsib `package.json`-ist `scripts` alt nime `dev` ja käivitab selle taga oleva käsu, praegu `vite`.
`npx tööriist`	käivitab tööriista ilma seda projekti lisamata.
`npm init -y`	teeb tühjale kaustale uue `package.json`-i. Nii algab iga uus Node'i projekt, ka meie backend.

Kausta `node_modules` Gitti ei panda. Seal on kümneid tuhandeid faile ja `npm install` teeb selle niikuinii uuesti.

JavaScript ilma brauserita

Siiani jooksis sinu JavaScript alati brauseris. Node'iga saab seda käivitada otse terminalis ja `console.log` kirjutab siis terminali, mitte konsooli:

```
node tere.js          # käivitab faili üks kord
node --watch tere.js  # käivitab uuesti iga kord, kui faili salvestad
```

Teist kuju kasutame backendiga, sest siis ei pea pärast iga muudatust serverit käsitsi uuesti käivitama.

Git

Gitist tunned juba kolme käsku. Kõige kasulikum on aga `git status`, mida me pole veel kasutanud. See ütleb, mis on pärast viimast commit'i muutunud ja mis on juba valmis salvestamiseks. Hea harjumus on vaadata `git status`-t enne ja pärast iga `git add`-i.

Käsk	Mida teeb
`git status`	mis on muutunud, mis on lisatud, mis on uus
`git diff`	näitab rida-realt, mida sa muutsid (välja saab `q`-ga)
`git add .`	võtab kõik muudatused järgmisesse commit'i
`git commit -m "…"`	salvestab need ajalukku koos sõnumiga
`git log --oneline`	näitab ajalugu, üks commit rea kohta

Serverid, pordid ja miks terminal „jäi kinni“

Kui käivitad `npm run dev`, siis käsk ei lõpe. Terminal ei näita uut `PS …>` rida, sest server töötab ja ootab brauseri päringuid. See ei ole viga. Terminal on lihtsalt hõivatud ja järgmise käsu jaoks on vaja uut terminali.

Aadressis `http://localhost:5173` tähendab `localhost` „see sama arvuti“ ja `5173` on **port**. Arvuti on nagu kortermaja ja port on korteri number: maja on üks, aga programme, kes seal elavad, on palju. Ühes korteris saab elada ainult üks programm korraga. Kui teine tahab samasse, tuleb viga `EADDRINUSE`.

Järgmisest tunnist on meil kaks serverit korraga. Seepärast hoia terminalid nii:

Terminal	Kaust	Mis seal töötab
1	`sookla-arvustused`	`npm run dev`, frontend aadressil `:5173`
2	`sookla-arvustused\server`	`node --watch server.js`, backend aadressil `:3000`
3	ükskõik kus	`git`, `npm install` ja kõik muu

Kui terminal ütleb midagi hirmsat

Punane tekst ei tähenda, et oled midagi ära lõhkunud. Veateade on terminali viis öelda, mis valesti läks, ja tavaliselt ütleb ta seda üsna täpselt. Loe viimased read läbi, sest põhjus on peaaegu alati seal.

Terminal ütleb	Mida see tähendab ja mida teha
`… is not recognized as the name of a cmdlet` või `command not found`	Sellist käsku ei ole. Kontrolli kirjapilti. Kui see on `npm` või `node`, sulge VS Code ja ava uuesti.
`running scripts is disabled on this system`	PowerShell ei luba npm-i käivitada. Ava terminali menüüst (nool + kõrval) **Command Prompt** ja proovi seal. Kui ka see ei aita, ütle mulle.

Terminal ütleb	Mida see tähendab ja mida teha
`ENOENT: no such file or directory … package.json`	Oled vales kaustas. `pwd`, siis `ls` ja `cd` õigesse projekti.
`Cannot find path … because it does not exist`	Sellist kausta pole. Kasuta Tab-klahvi, siis ei saa kirjaviga tekkida.
`fatal: not a git repository`	Oled kaustas, kus Giti pole. Mine projekti kausta.
`EADDRINUSE` või `Port 5173 is in use`	Server juba töötab teises terminalis. Leia see üles või kasuta Vite'i pakutud uut aadressi.
Uut rida ei tule, kursor vilgub	Programm töötab (näiteks server). Nii peabki. Peata Ctrl+C-ga.

> **Lühidalt:** vaata, kus sa oled (`pwd`), mis seal on (`ls`) ja mida terminal tegelikult ütles. Need kolm asja lahendavad suurema osa probleemidest.